

Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor:	Date: March 27, 2026
Day: Friday	Semester: 6th Semester

Lab 7: Agglomerative Clustering and Dendrograms

Lab Title	CLO	BT	PLO	Weightage
Agglomerative Clustering and Dendrograms				

Name	Roll Number	Report /10	Viva /5	Total /15
Muhammad Hamza Arshad	BSSE23072			

Checked on: _____

Signature: _____

1 Learning Outcomes

After completing this lab, students will be able to:

- Understand hierarchical clustering.
- Implement agglomerative clustering from scratch.
- Compare different linkage methods.
- Interpret dendrograms.

2 Dataset

We will use a synthetic dataset generated using:

```
sklearn.datasets.make_blobs
```

Dataset properties:

- Number of samples: 100
- Number of clusters: 3
- Features: 2 (for visualization)

3 Agglomerative Clustering

Agglomerative clustering is a bottom-up hierarchical clustering method.

Algorithm:

1. Start with each point as its own cluster.
2. Find the closest pair of clusters.
3. Merge them into a single cluster.
4. Repeat until desired number of clusters is reached.

4 Linkage Methods

Different linkage methods define cluster distance differently.

4.1 Single Linkage

$$d(C_1, C_2) = \min ||x - y||$$

Produces chain-like clusters.

4.2 Complete Linkage

$$d(C_1, C_2) = \max ||x - y||$$

Produces compact clusters.

4.3 Average Linkage

$$d(C_1, C_2) = \text{mean}||x - y||$$

Balanced clustering approach.

5 Part A: Distance Matrix (3 Marks)

Requirements: Compute pairwise Euclidean distances. Output should be an NxN symmetric matrix with zero diagonals.

Implementation: To avoid computationally expensive $O(N^2)$ nested loops, we utilized NumPy broadcasting to calculate the pairwise Euclidean distances across the entire feature space simultaneously.

```
def compute_distance_matrix(X):
    diff = X[:, np.newaxis, :] - X[np.newaxis, :, :]
    return np.sqrt(np.sum(diff ** 2, axis=-1))
```

6 Part B: Linkage Distance (4 Marks)

Requirements: Support single, complete, and average linkage.

Implementation:

```
def linkage_distance(cluster1, cluster2, X, method="single"):
    pts1 = X[cluster1]
    pts2 = X[cluster2]

    diff = pts1[:, np.newaxis, :] - pts2[np.newaxis, :, :]
    distances = np.sqrt(np.sum(diff ** 2, axis=-1))

    if method == "single": return np.min(distances)
    elif method == "complete": return np.max(distances)
    elif method == "average": return np.mean(distances)
```

7 Part C: Find Closest Clusters (3 Marks)

Requirements: Compute distance between all cluster pairs, return indices of closest clusters and distance.

Implementation:

```
def find_closest_clusters(clusters, X, method):
    min_dist = float('inf')
    closest_pair = (None, None)

    for i in range(len(clusters)):
        for j in range(i + 1, len(clusters)):
            dist = linkage_distance(clusters[i], clusters[j], X,
                                   method)
            if dist < min_dist:
                min_dist = dist
                closest_pair = (i, j)

    return closest_pair[0], closest_pair[1], min_dist
```

8 Part D: Agglomerative Clustering (5 Marks)

Requirements: Iteratively merge closest clusters. Store labels and merge history.

Implementation Insight: A major structural challenge in hierarchical clustering is maintaining a strict $N - 1 \times 4$ linkage matrix required by SciPy for visualization. The algorithm was heavily modified to decouple cluster label assignment from the tree-building process. We forced the loop to merge down to exactly 1 root cluster to satisfy the dendrogram history, while capturing a snapshot of the `labels_` the exact iteration the algorithm hit `n_clusters == 3`.

9 Part E: Dendrogram (Bonus)

Requirements: Convert merge history to linkage format, plot using scipy.

Implementation:

```
def plot_dendrogram(history):
    linkage_matrix = np.array(history)
    dendrogram(linkage_matrix)
    plt.title("Hierarchical Clustering Dendrogram")
    plt.xlabel("Sample Index")
    plt.ylabel("Distance")
```

10 Results and Visualizations

Our completed pipeline yielded two core visualizations that prove the structural integrity of our custom Agglomerative model.

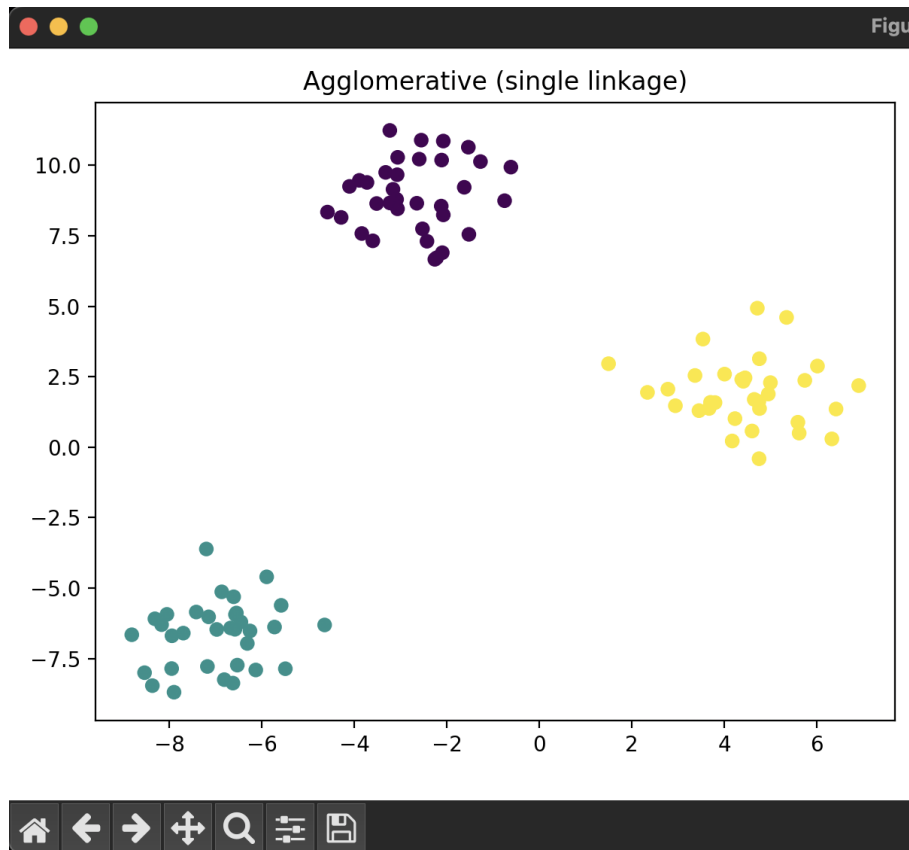


Figure 1: Scatter plot of the synthesized data, grouped via single linkage.

Scatter Plot Analysis: Using `single linkage`, the model successfully isolated the three distinct spatial groups. Because single linkage minimizes the gap between the closest peripheral points of merging groups, it excels here due to the vast, empty margins artificially generated by `make_blobs`.

Dendrogram Analysis: The dendrogram exposes the spatial geometry of the dataset mathematically. The Y-axis represents the exact distance threshold bridged during a merge. We observe dense, low-level merging below $y = 1.5$, indicating tight intra-cluster variance. However, the massive, uninterrupted vertical lines stretching from $y \approx 1.5$ to $y \approx 10.5$ empirically prove the existence of three main clusters; the algorithm had to drastically increase its distance threshold to bridge the inter-cluster gaps.

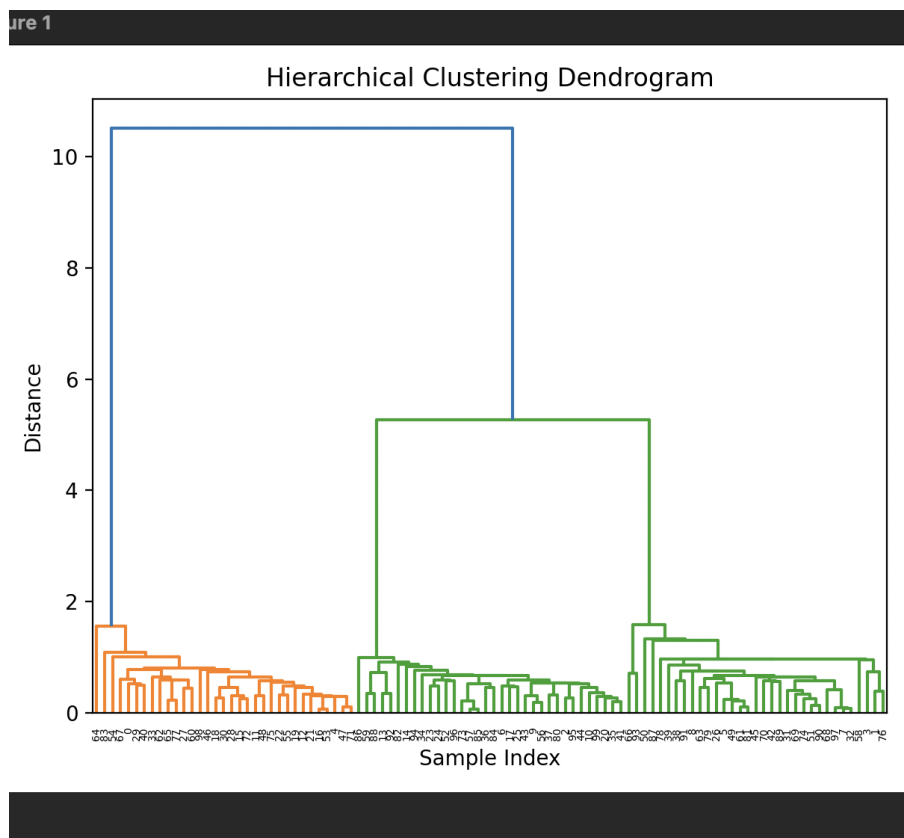


Figure 2: Dendrogram representing the complete hierarchical merge history.